LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [5] – [Relasi antar Kelas]



NAMA : Abdullah Ahmad Izzah

NIM : 103132430024

KELAS : S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram kelas merupakan diagram yang digunakan untuk menggambarkan kelas-kelas dalam suatu sistem beserta hubungan antar kelas tersebut. Diagram kelas memiliki beberapa elemen utama, yaitu nama kelas, atribut, method, dan relasi antar kelas. Nama kelas ditampilkan pada bagian atas, atribut pada bagian tengah, serta method pada bagian bawah. Diagram kelas juga menggunakan simbol tertentu untuk menunjukkan access modifier, seperti tanda + untuk public, tanda - untuk private, dan tanda # untuk protected.

1. [Asosiasi]

   File asosiasi Dosen.java

```
package guided.asosiasi;

public class Dosen {

    private String kodeDosen;
    private String namaDosen;

    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");
```

```
        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

File asosiasi Mahasiswa.java

```
package guided.asosiasi;

public class Mahasiswa {

    private String nim;
    private String nama;
    private int nilai;

    //Setter
    public void setNama(String nama) {
        this.nama = nama;
    }
    public void setNim(String nim) {
        this.nim = nim;
    }
    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    //Getter
    public String getNama() {
        return this.nama;
    }
    public String getNim() {
        return this.nim;
    }
    public int getNilai() {
        return this.nilai;
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project_Modu
Nim :130118383
Nama :Budi
Nilai :90
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

2. [Agregasi]

File Agregasi Jurusan.java

```
package guided.agregasi;
public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i=0; i<jumlahMahasiswa; i++){
            System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }
    }

    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    //Setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

File Agregasi Mahasiswa.java

```
package guided.agregasi;

public class Mahasiswa {
   private String NIM, Nama;

   public Mahasiswa(String NIM, String Nama){
      this.NIM = NIM;
      this.Nama = Nama;
   }

   //Getter
   public String getNIM(){
      return NIM;
   }
   public String getNama(){
      return Nama;
   }

   //Setter
   public void setNIM(String NIM){
      this.NIM = NIM;
   }
   public void setNama(String Nama){
      this.Nama = Nama;
   }
}
```

File Agregasi Main.java

```
package com.mycompany.project_modul_5;
import guided.agregasi.*;

public class Project_Modul_5 {
   public static void main(String[] args) {
     runGuided2();
   }

   public static void runGuided2(){
      Mahasiswa m1 = new Mahasiswa("103132430024","Ijah");
      Mahasiswa m2 = new Mahasiswa("103132430025", "ronaldo");

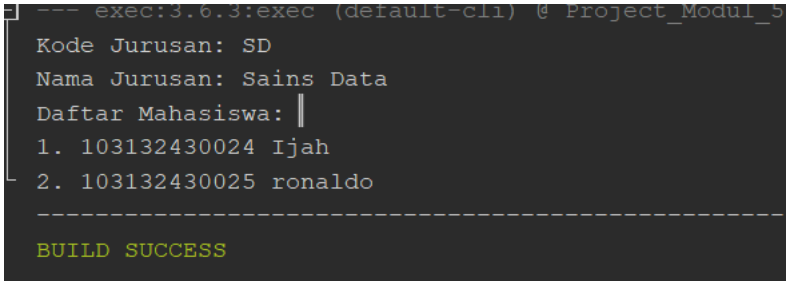
      Jurusan prodi = new Jurusan("SD", "Sains Data");

      prodi.addMahasiswa(m1);
      prodi.addMahasiswa(m2);
```

```
        prodi.displayMahasiswa();

    }
}
```

Output :



B. Unguided

1. Soal Unguided 1

   Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.
   Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

   Jawaban :

   File RincianHarga.java

```
package unguided;

public class RincianHarga {

    private double hargaDasar;
    private double pajak;

    private static final double TARIF_PAJAK = 0.10;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * TARIF_PAJAK;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }
```

```
    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

File ItemPesanan.java

```
package unguided;

public class ItemPesanan {

    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

File Pesanan.java

```
package unguided;

public class Pesanan {

    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }
```

```
    public void tambahItem(ItemPesanan item) {
        if (item != null && jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Pesanan penuh atau item tidak valid.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);

        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println((i + 1) + ". "
                    + item.getNamaItem()
                    + " - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar()
                    + ", Pajak: " + rincian.getPajak() + ")");
        }
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem.clone();
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

File Kasir.java

```
package unguided;

public class Kasir {

    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }
```

```
    public void prosesPembayaran(Pesanan pesanan) {
        double subtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftarItem = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian
                    = daftarItem[i].getRincian();

            subtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        double totalBayar = subtotal + totalPajak;

        System.out.println();
        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println();
        System.out.println("Pesanan        : Meja "
                + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : "
                + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + subtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

File Main.java

```
package unguided;

public class Main {

    public static void main(String[] args) {

        // Membuat objek kasir
        Kasir kasir = new Kasir("Budi");

        // Membuat objek item secara independen
        ItemPesanan item1
                = new ItemPesanan("Iced Americano", 18000);

        ItemPesanan item2
                = new ItemPesanan("Beef Croissant", 25000);

        ItemPesanan item3
```

```
            = new ItemPesanan("Matcha Latte", 22000);

        // Membuat pesanan untuk meja 4 dengan kapasitas 5 item
        Pesanan pesanan = new Pesanan(4, 5);

        // Memasukkan item ke dalam pesanan (agregasi)
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // Menampilkan rincian pesanan
        pesanan.tampilkanPesanan();

        // Memproses pembayaran (asosiasi)
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project_Modul_5 ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi

Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program ini terdiri dari lima class yang saling berhubungan, yaitu RincianHarga, ItemPesanan, Pesanan, Kasir, dan Main. Seluruh atribut dibuat dengan modifier private sebagai penerapan enkapsulasi, sehingga atribut tidak dapat diakses secara langsung dari luar class. Penggunaan constructor dengan keyword this digunakan untuk membedakan parameter dan atribut yang memiliki nama sama.

Class RincianHarga bertugas menghitung pajak sebesar 10% dari harga dasar dan menentukan harga total setiap menu. Class ItemPesanan menyimpan nama menu

serta objek RincianHarga yang dibuat langsung di dalam constructornya. Hubungan antara kedua class tersebut merupakan komposisi karena rincian harga menjadi bagian internal dari item pesanan.

Class Pesanan menyimpan nomor meja, array daftarItem, dan jumlah item yang telah dimasukkan. Method tambahItem() digunakan untuk memasukkan objek item yang sudah dibuat sebelumnya ke dalam array. Hubungan ini merupakan agregasi karena objek ItemPesanan dibuat secara independen sebelum dimasukkan ke dalam objek pesanan. Method tampilkanPesanan() kemudian menampilkan nama menu, harga dasar, pajak, dan harga total setiap item.

Class Kasir menyimpan nama petugas dan mempunyai method prosesPembayaran() yang menerima objek Pesanan sebagai parameter. Method tersebut menjumlahkan harga dasar dan pajak seluruh item untuk menghasilkan subtotal, total pajak, dan total pembayaran. Hubungan antara Kasir dan Pesanan merupakan asosiasi karena objek pesanan hanya digunakan ketika method pembayaran dipanggil dan tidak disimpan sebagai atribut pada class Kasir.

Pada class Main, program membuat objek kasir dan tiga objek item, kemudian membuat pesanan untuk meja 4. Ketiga item dimasukkan ke dalam array pesanan menggunakan method tambahItem(). Setelah rincian pesanan ditampilkan, kasir memproses pembayaran dan menghasilkan subtotal sebesar Rp65.000, pajak Rp6.500, serta total pembayaran Rp71.500.

C. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa relasi antar kelas dalam Pemrograman Berorientasi Objek dapat diterapkan sesuai dengan kebutuhan suatu sistem. Komposisi digunakan pada hubungan ItemPesanan dengan RincianHarga, karena objek rincian harga dibuat di dalam constructor item pesanan. Agregasi diterapkan pada hubungan Pesanan dengan ItemPesanan, karena objek item dibuat secara terpisah sebelum dimasukkan ke dalam array pesanan. Sementara itu, asosiasi diterapkan pada hubungan Kasir dengan Pesanan, karena objek pesanan digunakan sebagai parameter dalam proses pembayaran. Program berhasil menampilkan rincian pesanan dan menghitung total pembayaran dengan benar. Praktikum ini juga memperkuat pemahaman mengenai enkapsulasi, penggunaan constructor, array objek, dan interaksi antar kelas dalam Java.